Laboratory No. 3: Training Neural Networks

Training Loops, Imbalanced Data, and Weight Initialization

Lab 3: Training Neural Networks

Faculty of Mathematics and Information Science
Warsaw University of Technology

March 9, 2026

Laboratory Rules

- All places you need to implement something are highlighted
- To check every part: run `python ./part_1.py`, `python ./part_2.py`, `python ./part_3.py`
- `helpers.training_utils` is not important - only read function descriptions if needed
- On this Lab please don't use chat

What You Will Do

1. **Part 1:** Implement basic training loop with model architecture, data loaders, training, and validation
2. **Part 2:** Implement three methods for handling imbalanced datasets (oversampling, class weights, weighted sampling)
3. **Part 3:** Implement weight initialization strategies and hyperparameter tuning

Part 1: Implementing Data Loaders

How to create DataLoaders from train/val/test arrays:

1. **TensorDataset** — wrap NumPy arrays as PyTorch tensors:
   `train_ds = TensorDataset(torch.from_numpy(X_train).float(), torch.from_numpy(y_train).long())`
2. **DataLoader** — batch and shuffle:
   `train_loader = DataLoader(train_ds, batch_size=64, shuffle=True)`
   Use `shuffle=True` for training (random order each epoch), `shuffle=False` for validation and test
3. Return `train_loader`, `val_loader`, `test_loader` (same pattern for val and test with `shuffle=False`)

Part 1: Iterating Over Batches

DataLoader iteration:

- `for inputs, targets in train_loader:` - iterates through batches
- `inputs`: shape (batch_size, input_features) - input data for current batch
- `targets`: shape (batch_size,) - true class labels for current batch
- **Why Lab 2 used `.squeeze()` and Lab 3 does not:** In Lab 2 Part 2 (*binary classification*) the model has `output_size=1`: one logit per sample, shape `(batch_size, 1)`. `BCEWithLogitsLoss` expects predictions and targets of shape `(batch_size,)`, so `squeeze()` removes the trailing dimension. (Lab 2 Part 3 used *regression* with `MSELoss` and the same `(batch_size, 1)` → `squeeze()` for the same reason.) In Lab 3 we have *multi-class classification*: model outputs `(batch_size, num_classes)`; `CrossEntropyLoss` and `outputs.max(dim=1)` expect that shape, so no `squeeze`.

Part 1: Calculating Accuracy

Steps:

1. Get predictions: `_, predicted = outputs.max(dim=1)` - get class with highest probability
2. Compare with targets: `(predicted == targets)` - boolean tensor
3. Count correct: `(predicted == targets).sum().item()` - number of correct predictions
4. Calculate accuracy: `correct / total`

Part 1: Calculating Loss - Why Multiply and Divide?

Step 1: Convert average to sum in single Batches

- `loss.item() * inputs.size(0)` - multiply average by batch size
- This gives us **total loss** for all samples in this batch

Step 2: Sum across all batches

- `running_loss += ...` accumulates total loss from all batches
- Now we have sum of losses from **all samples** in dataset

Step 3: Convert sum to average in entire dataset

- `train_loss = running_loss / len(train_loader.dataset)`
- Divide by total number of samples to get average per sample

Part 1: Why model.train() and model.eval()?

model.train():

- Enables dropout layers
- Batch normalization uses batch statistics
- Required for training

model.eval():

- Disables dropout (uses all neurons)
- Batch normalization uses running statistics
- Required for validation/test

Why torch.no_grad()?

- Disables gradient computation - saves memory
- Speeds up evaluation - no need to track gradients
- Required for validation/test - we're not training

In the training loop: `evaluate()` calls `model.eval()` and `torch.no_grad()` inside, so the model switches to evaluation mode for validation. The loop calls `model.train()` at the start of each epoch, so the model is back in training mode for the next epoch.

Part 1: GPU/CPU Device Handling

PyTorch can automatically handles GPU/CPU computation

Device detection:

- `torch.cuda.is_available()` - Check if GPU is available
- `torch.device('cuda' if torch.cuda.is_available() else 'cpu')` - Select device
- `torch.cuda.get_device_name(0)` - Get GPU name if available

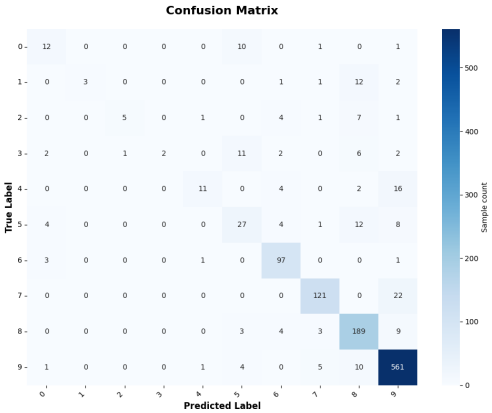
Moving tensors and models:

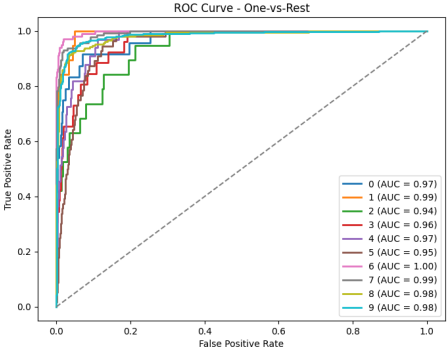
- `model.to(device)` - Move model to GPU/CPU
- `inputs.to(device)`, `targets.to(device)` - Move data to same device as model

Part 1: Help Info for This Lab

1. **model.train() / model.eval()** - Switch between training and evaluation modes. Always use `model.train()` before training loop, `model.eval()` before validation
2. **torch.no_grad()** - Context manager that disables gradient computation. Use during validation to save memory and speed up evaluation
3. **optimizer.zero_grad()** - Clears gradients from previous iteration. Must call before each `loss.backward()` to avoid gradient accumulation
4. **loss.backward()** - Computes gradients using automatic differentiation. Builds computational graph and calculates derivatives
5. **optimizer.step()** - Updates model parameters using computed gradients. Applies the optimization step (e.g., SGD update rule)
6. **loss.item()** - Extracts scalar value from tensor. Converts PyTorch tensor to Python float
7. **inputs.size(0)** - Gets batch size (first dimension of input tensor). Use to convert average loss to total loss
8. **outputs.max(dim=1)** - Returns (values, indices) tuple. Use indices for class predictions: `_, predicted = outputs.max(dim=1)`

Part 2: Evaluation — ROC, Confusion Matrix, F1





Confusion Matrix: Reveals misclassifications; affected by class imbalance.

ROC: High AUC (0.94–1.00) = strong separability; measures ranking of predicted probabilities.

Why F1: Balances precision and recall; more reliable than accuracy or ROC

Part 2: The Problem of Class Imbalance

What happens with imbalanced data?

- Model learns to predict majority class most of the time
- High overall accuracy but poor performance on minority classes
- Example: 90% class A, 10% class B $\rightarrow$ model predicts A always $\rightarrow$ 90% accuracy but 0% on class B

Part 2: Why Accuracy is Misleading

Problem with accuracy on imbalanced data:

- Accuracy = (correct predictions) / (total predictions)
- Model can achieve high accuracy by always predicting majority class
- Doesn't tell us about performance on minority classes

Better metrics:

- **F1-macro:** Average F1 score across all classes (treats all classes equally)
- **ROC-AUC:** Area under ROC curve (measures ability to distinguish classes)
- Both metrics are class-balanced and reveal true model performance

Part 2: Method 1 - Oversampling

Concept: Duplicate minority class samples to balance dataset.
Implementation:

- `ros = RandomOverSampler(sampling_strategy='auto')`
- `X_resampled, y_resampled = ros.fit_resample(X_train, y_train)`

Result: All classes have same count (matches majority class). **Warning:** Can cause overfitting (exact duplicates).

Part 2: Method 2 - Class-Weighted Loss

Concept: Modify loss to penalize minority class errors more.
Weight formula: weight $= \frac{\text{total_samples}}{\text{n_classes} \times \text{samples_per_class}}$
Example: Class A (90 samples) $\rightarrow$ weight 0.56, Class B (10 samples) $\rightarrow$ weight 5.0 (9$\times$ more penalty!)
Implementation:

- Use `_calculate_class_weights(y_train)` to get weights
- `nn.CrossEntropyLoss(weight=weights_tensor)`

Part 2: Method 3 - Weighted Random Sampling

Concept: Balance each batch by sampling minority classes more frequently.
Implementation:

- Use `_calculate_class_weights(y_train)` to get class-to-weight dict
- Map each sample to its class weight: `[class_to_weight[c] for c in y_train]`
- `WeightedRandomSampler(weights, len(weights))`
- `DataLoader(dataset, sampler=sampler, shuffle=False)`

Difference: Oversampling changes dataset, class weights change loss, weighted sampling changes batches.

Part 2: Help Info for This Part

1. **_calculate_class_weights(y_train)** - Helper function that returns:
   - `weights_tensor`: PyTorch tensor for `CrossEntropyLoss(weight=...)`
   - `class_to_weight_dict`: Dictionary for mapping samples to weights in WeightedRandomSampler
2. **RandomOverSampler** - Use `sampling_strategy='auto'` to balance all classes
3. **Class-weighted loss** - Get weights from `_calculate_class_weights()`, pass tensor to `CrossEntropyLoss(weight=weights)`
4. **WeightedRandomSampler** - Get dict from `_calculate_class_weights()`, create sample weights array, use in DataLoader with `shuffle=False`

Part 3: Why Weight Initialization Matters

Good initialization:

- Weights in appropriate range
- Maintains gradient flow
- Speeds up convergence

Part 3: Xavier (Glorot) Initialization

Xavier Uniform:

- Designed for tanh/sigmoid activations
- Weights sampled from uniform distribution; range $[-\sqrt{6/(n_{in}+n_{out})}, \sqrt{6/(n_{in}+n_{out})}]$
- Keeps variance of activations and gradients similar across layers

Code: loop over `model.modules()`, for each `nn.Linear` apply init then zero bias.

```
def init_model_xavier(model: nn.Module) -> None:
    for module in model.modules():
        if isinstance(module, nn.Linear):
            nn.init.xavier_uniform_(module.weight)
            if module.bias is not None:
                nn.init.constant_(module.bias, 0.0)
```

Part 3: Kaiming (He) Initialization

Kaiming Uniform:

- Designed for ReLU; range $[-\sqrt{6/n_{in}}, \sqrt{6/n_{in}}]$ (n_{in} = layer input size)
- Accounts for ReLU zeroing half the activations; better for deep nets with ReLU

Usage: `nn.init.kaiming_uniform_(module.weight, nonlinearity='relu')`

- **nonlinearity='relu':** tells PyTorch that the next layer is ReLU. The scaling of weights is derived for that activation (ReLU zeros half the values, so the formula uses a different gain). Use `'leaky_relu'` or `'linear'` if the layer is followed by a different activation.

Part 3: Simple Initializations (Uniform & Normal)

Uniform: weights in $[-0.1, 0.1]$; simple baseline. **Normal:** weights $\mathcal{N}(0, 0.1)$; different distribution. **Bias:** always zero for all init methods.

```
def init_model_uniform(model: nn.Module) -> None:
    for module in model.modules():
        if isinstance(module, nn.Linear):
            nn.init.uniform_(module.weight, -0.1, 0.1)
            if module.bias is not None:
                nn.init.constant_(module.bias, 0.0)

def init_model_normal(model: nn.Module) -> None:
    for module in model.modules():
        if isinstance(module, nn.Linear):
            nn.init.normal_(module.weight, 0.0, 0.1)
            if module.bias is not None:
                nn.init.constant_(module.bias, 0.0)
```

Part 3: Hyperparameter Tuning

What are hyperparameters?

- Parameters not learned during training
- Set before training starts
- Examples: learning rate, batch size, momentum, weight initialization

Grid search:

- Test all combinations of hyperparameters
- Compare results (F1-macro score, accuracy)
- Find best configuration

Help Info for Part 2 and 3

1. **_calculate_class_weights(y_train)** returns tuple:
   - `weights_tensor`: PyTorch tensor for `CrossEntropyLoss(weight=...)`
   - `class_to_weight_dict`: Dictionary for mapping samples in WeightedRandomSampler
2. **Oversampling:** `RandomOverSampler(sampling_strategy='auto')` then `fit_resample()`
3. **Class weights:** Use first element of tuple: `weights, _ = _calculate_class_weights(y_train)`
4. **Weighted sampler:** Use second element: `_, class_to_weight = _calculate_class_weights(y_train)`, then map samples
5. **Weight init:** Iterate with `model.modules()`, check `isinstance(module, nn.Linear)`, apply init to `module.weight`
6. **Hyperparameter lists:** Return simple Python lists, e.g., `return [0.001, 0.01]` for learning rates